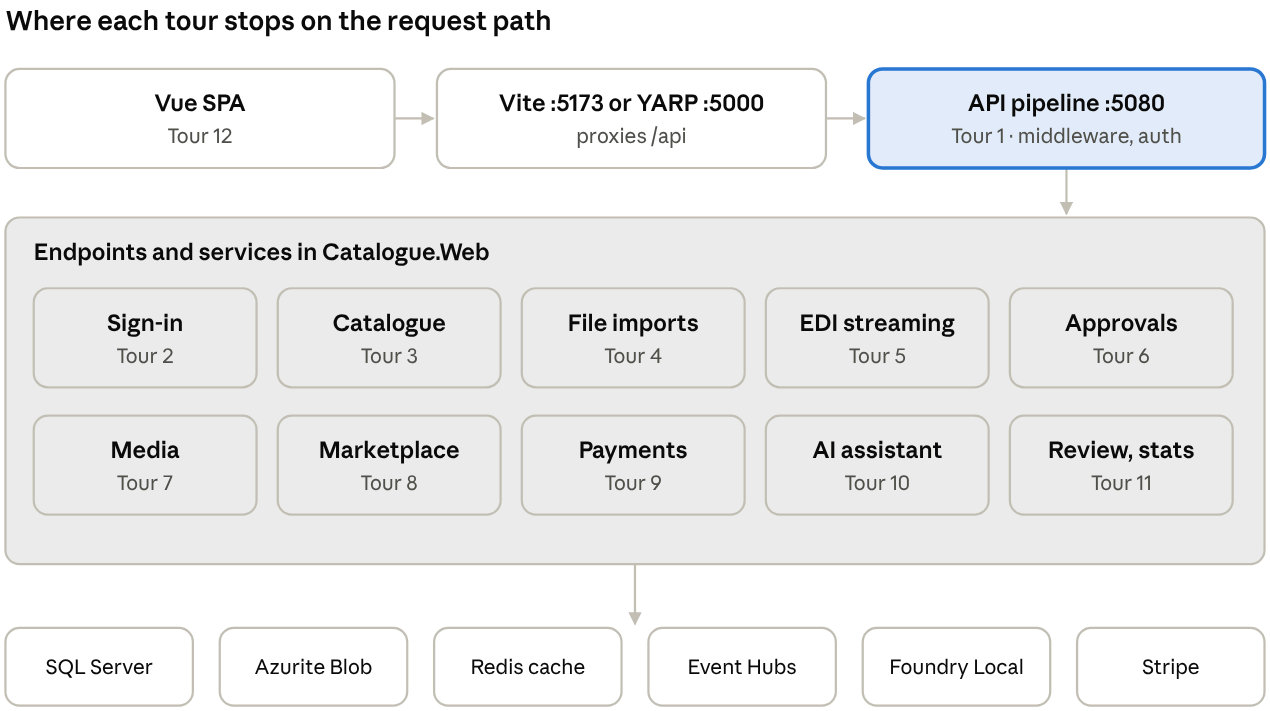
Catalogue Management System — Debugging Guide

Oct 5, 2026 · @Rupam

Twelve guided tours put breakpoints at exactly the lines where each part of the application does its work, from the HTTP pipeline to the AI agent. Every stop gives the file and line, what to trigger in the app, and what to inspect. Sections 1–2 set up the debugger; each tour after that stands on its own.



request path · 12 tours

Every request enters through the SPA and the API pipeline (tours 12 and 1); the ten feature areas in the middle each have their own tour, and all of them read and write the stores along the bottom.

1. Setup: run everything under the debugger

Open the repository root in VS Code (C# Dev Kit + the built-in JavaScript debugger), Visual Studio 2022 17.14+ or Rider. The launch configurations below are in `.vscode/launch.json` and `.vscode/tasks.json`; pick them in the Run and Debug panel (Ctrl+Shift+D).

Configuration	What it starts	Use it for
Full stack: API + Gateway + SPA	API on :5080, YARP gateway on :5000, Vite on :5173 in an Edge debug window	Following a click from the browser to SQL and back
API + SPA	API + Vite + Edge	Most tours in this guide
.NET: Launch API (Catalogue.Web)	API only, opens Swagger	Backend tours driven from Swagger, curl or a `.http` file
.NET: Launch Gateway (YARP :5000)	Gateway only	Routing, rate limits, gateway timeouts
.NET: Launch Worker	`Catalogue.Workers` (Service Bus and Event Hub consumers)	Only when `Worker:InProcess=false`; by default the API runs the worker itself
.NET: Attach to a running process	Pick `Catalogue.Web.exe`	A process you already started with `dotnet run`
Vue: Debug SPA in Edge / Chrome	Vite + a browser with breakpoints in `.vue` and `.ts` files	Frontend tour
.NET: Debug unit tests	`dotnet test` with a filter, waits for the debugger	Stepping through a test; Test Explorer's *Debug Test* is easier

Before you press F5

1. Infrastructure: `docker compose up -d azurite redis eventhubs otel-dashboard` (Blob storage, cache, Event Hubs emulator, traces). Without them the app still runs: storage falls back to `./.storage`, the cache to in-process, the hub to in-memory.
2. SQL Server LocalDB must exist (`sqllocaldb info`). The database is created and seeded on the first start.
3. Foundry Local for the agent tours: `foundry server start`. The API loads the model on first use.
4. Sign in as `admin` / `admin123` (everything), `buyer` / `buyer123` (basket, orders, payments) or `seller` / `seller123` (seller portal).

Visual Studio / Rider: set *Catalogue.Web* as the startup project with profile `http`; for several processes use *Configure Startup Projects* → *Multiple* (Web + Gateway), then start Vite with `npm run dev` in `web/catalogue-ui` and debug it in the browser's DevTools.

Driving requests without the browser. Every backend tour can be started from Swagger (`http://localhost:5080/swagger`, *Authorize* with the token from `POST /api/auth/login`) or from `src/Catalogue.Web/Catalogue.Web.http` in VS Code / Visual Studio. This matters: the SPA's API client (`web/catalogue-ui/src/api.ts:4`) has a **60-second timeout and retries GET requests** on timeouts, so a long pause at a breakpoint makes the browser send the request again and you hit the breakpoint a second time. For long investigations, drive the request from Swagger or the `.http` file instead.

2. Breakpoint techniques used in the tours

Each tour lists breakpoints as `file:line`, with the variable or expression to watch. Line numbers match the code on `main` at the time of writing; if a line moved, search for the quoted code.

Technique	How (VS Code)	When the tours use it
Line breakpoint	Click left of the line number, or F9	Most stops
Conditional breakpoint	Right-click the dot → *Edit Breakpoint* → *Expression*, e.g. `id == 42`	Code that runs for every row or every request (bulk import, list queries)
Hit count	Same menu → *Hit Count*, e.g. `100`	Loops over thousands of rows: stop on the 100th
Logpoint	Same menu → *Log Message*, e.g. `order {o.Number} -> {next}`	Watching a flow without pausing it: background workers, streams, the agent run
Exception breakpoint	Run and Debug → *Breakpoints* → tick *All Exceptions* (or *User-Unhandled*)	Finding where a 4xx/5xx really starts before `GlobalExceptionHandler` turns it into a problem response

Technique	How (VS Code)	When the tours use it
Breakpoint in a lambda	Put the cursor inside the lambda body and press F9; VS Code asks *line* or *lambda* — choose the lambda	Minimal API handlers are lambdas: `api.MapGet("/products", async (...) => { ... })`. A breakpoint on the `MapGet` line stops once at startup, not per request
Async call stack	Call Stack panel → the async frames show which handler awaited what	Following `await` chains through services
Data inspection	Watch window, Debug Console (evaluate `db.ChangeTracker.DebugView.LongView`)	Seeing what EF Core is about to save
Step into / over / out	F11 / F10 / Shift+F11	Walking a single request

Three traps to know before you start

1. **Request timeouts.** The API cancels requests after 30 s (15 min for uploads) and the gateway after 60 s. ASP.NET Core's timeout middleware is skipped while a debugger is attached to *that* process, so the API is fine. But the gateway (if you go through :5000), the Vite proxy and the SPA's 60 s axios timeout still apply. The cancellation shows up as `OperationCanceledException` on the `CancellationToken ct` the handler received.
2. **Retries.** The SPA retries GET requests that time out, so a long pause produces duplicate hits. Drive long investigations from Swagger or `Catalogue.Web.http`.
3. **The 30-minute access token.** A JWT expires after 30 minutes. If you sit at a breakpoint longer, the next request returns 401 and the SPA refreshes the token (tour 2 shows how). From Swagger, sign in again.

Just My Code is on in every configuration, so stepping skips framework code. Turn it off (`"justMyCode": false`) only when you want to step into ASP.NET Core, EF Core or Agent Framework sources.

3. Tour 1 — Startup and the request pipeline

Every request passes the same middleware in the same order before it reaches an endpoint; this tour stops at each one. Start **.NET: Launch API** and call any authenticated endpoint, e.g. `GET /api/auth/me` from Swagger.

Part A — startup (stops once)

#	Breakpoint	What to look at
1	`src/Catalogue.Web/Program.cs:20` — `var builder = WebApplication.CreateBuilder(args);`	Configuration sources: inspect `builder.Configuration.GetDebugView()` in the Debug Console to see which appsettings value won
2	`src/Catalogue.Infrastructure/DependencyInjection.cs:302` — `StorageSelection.Resolve`	Whether Blob (Azurite) or disk storage is chosen, and why
3	`src/Catalogue.Web/Program.cs:164` — `await db.SeedAsync();`	Step in (F11): `EnsureCreated`, then `SchemaEvolver.EvolveAsync` (`DependencyInjection.cs:152`) adds missing tables, columns and indexes
4	`src/Catalogue.Web/Program.cs:165` — `IdentitySeed.SeedAsync`	Demo companies and users being created on an empty database
5	`src/Catalogue.Web/Program.cs:188` — `var api = app.MapGroup("/api");`	Routes are only *registered* here. Step over the `Map(api)` lines: no handler runs

Part B — one request, middleware by middleware (order set in `Program.cs:147-159` and `ServiceDefaults.cs:146`)

#	Breakpoint	What to look at
1	`src/Catalogue.ServiceDefaults/Correlation.cs:15` — `CorrelationIdMiddleware.InvokeAsync`	`inbound` (line 17): the `X-Correlation-Id` the client sent, or a new one; it is copied to the response (line 25) and the trace (line 26)
2	`src/Catalogue.Web/Program.cs:106` — `AddJwtBearer` options	Set a breakpoint inside to see the token rules; at request time put an **exception breakpoint** on `SecurityTokenExpiredException` to watch an expired token being rejected
3	`src/Catalogue.Web/Security/Identity.cs:184` — `CurrentUser.UserId`	How claims from the JWT become `me.UserId`, `me.OrganizationId`, `me.SupplierId` inside handlers
4	`src/Catalogue.Web/Security/HttpActorContext.cs:8` — `ActorId`	The name written into every audit event
5	The endpoint's lambda (e.g. `src/Catalogue.Web/Endpoints/AuthEndpoints.cs:136`)	Parameters already bound: route, query, body, DI services

#	Breakpoint	What to look at
6	`src/Catalogue.ServiceDefaults/GlobalExceptionHandler.cs:25` — `Classify(exception, context)`	Only when a handler throws: which exception maps to which status (`KeyNotFoundException` → 404, `InvalidOperationException` → 409, `ArgumentException` → 400)

Try it: call `GET /api/products/999999999` (404 returned by the handler) and then `GET /api/products?spec=NOPE:eq:1` (an `ArgumentException` thrown by the spec filter): the first never reaches stop 6, the second does. Compare the two response bodies: both carry `correlationId` and `traceId`, which you can search for in the log file under `src/Catalogue.Web/logs` and in the Aspire dashboard (`http://localhost:18888`).

Authorization failures (401/403) never reach a handler: the policy check in `UseAuthorization` (`Program.cs:158`) stops them first. To see why a call is forbidden, compare the roles in the token (`me.IsInRole(...)`) with the policy table in `docs/api/README.md`.

4. Tour 2 — Sign-in, token refresh and registration

A session is a 30-minute JWT kept in browser memory plus a rotating refresh token in an httpOnly cookie. Start **API + SPA** and sign in as `buyer` / `buyer123`.

Sign-in (`POST /api/auth/login`)

#	Breakpoint	What to look at
1	`src/Catalogue.Web/Endpoints/AuthEndpoints.cs:34` — `var key = ...ToLowerInvariant();`	Email or username, normalised
2	`AuthEndpoints.cs:39` — `if (user is null)`	Sign in with a wrong username: a dummy hash is still verified so timing doesn't reveal which accounts exist
3	`AuthEndpoints.cs:48` — `hasher.VerifyHashedPassword(...)`	`verdict`: Failed / Success / SuccessRehashNeeded

#	Breakpoint	What to look at
4	`AuthEndpoints.cs:51` — `user.LoginFailed(now)`	Enter a wrong password 5 times: watch `user.FailedLogins` climb and `LockoutEndUtc` get set, then stop 2's sibling at line 44 returns 423
5	`AuthEndpoints.cs:58` — company status check	Sign in as `pennine` / `pennine123`: the company is `PendingVerification`, so 403
6	`src/Catalogue.Web/Security/Identity.cs:74` — `TokenService.CreateAccessToken`	The claims put into the JWT: user id, roles, organization, customer and supplier ids
7	`Identity.cs:101` — `IssueRefreshTokenAsync`	Only a SHA-256 hash of the refresh token is stored; the raw value goes into the cookie (`Identity.cs:148`)

Refresh and theft detection (`POST /api/auth/refresh`)

The SPA refreshes the token shortly before it expires and on any 401. Set a browser breakpoint in `web/catalogue-ui/src/api.ts:15` (`refreshSession`) and a server breakpoint at `Identity.cs:113` (`RotateAsync`).

#	Breakpoint	What to look at
1	`Identity.cs:113` — `RotateAsync`	The cookie's token is looked up by hash
2	`Identity.cs:126` — `return (RefreshOutcome.Reused, ...)`	Replay an old cookie (copy `cat_rt` from DevTools before a refresh, then send it again with curl): reuse revokes the whole token family
3	`Identity.cs:129` — security-stamp check	After `POST /api/auth/change-password`, older refresh tokens fail here

#	Breakpoint	What to look at
4	`Identity.cs:134` — `IssueRefreshTokenAsync(user, token.FamilyId, ...)`	Rotation: the new token stays in the same family

Registration (`POST /api/auth/register`)

Break at `AuthEndpoints.cs:72` (`var errors = new Dictionary<...>`) and submit the Register form with a weak password: watch `errors` fill per field and come back as a ValidationProblem (line 80). With valid input, step through lines 86–98: the company is created as `PendingVerification`, then its first user with roles OrgAdmin + Buyer/Seller. Approval happens in tour 6′s neighbour, `OrganizationEndpoints.cs:41` (`/approve`), which creates the Supplier and CustomerAccount.

5. Tour 3 — Catalogue search, product detail, caching and pricing

The list endpoint turns filters into one SQL query over 7 million products, caches the count, and prices the page for the buyer. Open the Catalogue page as `buyer` and search "boiler".

#	Breakpoint	What to look at
1	`src/Catalogue.Web/Endpoints/ProductEndpoints.cs:74` — `var term = search.Trim();`	How the search term is classified (lines 76–79): GTIN → exact match, SKU-like → prefix, otherwise name/brand prefix
2	`ProductEndpoints.cs:110` — `query = ApplySpecFilter(query, f, refData);`	Add a spec filter in the UI (e.g. output kW range) and step in: each `spec=CODE:op:value` becomes an `Any(...)` over attribute values
3	`ProductEndpoints.cs:132` — `cache.GetOrCreateAsync(countKey, ...)`	First search: the factory lambda runs (`COUNT(*)`). Same search again within a minute: it doesn′t — the count comes from HybridCache
4	`ProductEndpoints.cs:137` — `var rows = await query.Skip(...)`	**Before** stepping over, evaluate `query.ToQueryString()` in the Debug Console to see the exact SQL EF Core will send

#	Breakpoint	What to look at
5	`ProductEndpoints.cs:157` — `pricing.QuoteAsync(...)`	Only with a customer: step into `src/Catalogue.Application/Pricing.cs:96`, then `PriceResolver.Resolve` (`Pricing.cs:31`) for the price-list chain (customer → tier → trade → list price)
6	`ProductEndpoints.cs:168` — `return Results.Ok(new PagedResult...)`	The DTOs sent to the browser, with signed image URLs

Product detail (`GET /api/products/{id}`) — open a product drawer.

#	Breakpoint	What to look at
1	`ProductEndpoints.cs:217` — `cache.GetOrCreateAsync(CacheTags.Product(id), ...)`	Cache hit vs miss: on a miss `LoadDetail` (line 387) runs with `AsSplitQuery()` (line 396) — one SQL query per included collection
2	`ProductEndpoints.cs:223` — `detail = detail with { Assets = ... }`	Media URLs are added per request (they expire), never cached
3	`ProductEndpoints.cs:231` — customer quote	Customer-specific prices are also never cached

Cache invalidation. Edit a product as `admin` (`PUT /api/products/{id}`) with breakpoints at `ProductEndpoints.cs:274` (`ValidateProduct`) and `ProductEndpoints.cs:306` (`RemoveByTagAsync(CacheTags.Products)`): every cached list, count and detail tagged `products` is dropped, so the next search runs the factory again (stop 3 above fires). The Redis layer is wrapped in `ResilientDistributedCache` (`src/Catalogue.Web/Infrastructure.cs:31`): stop Redis (`docker stop` on its container) and step through it to see a cache outage absorbed instead of failing the request.

Reference data (attribute definitions, category templates) comes from `ReferenceDataProvider` (`src/Catalogue.Infrastructure/Persistence/ReferenceDataProvider.cs:12`), an in-process snapshot that category and attribute writes invalidate.

6. Tour 4 — A CSV/EDI file import, end to end

An upload returns 202 immediately; a background worker in the same process then parses, validates and writes the file. Upload `samples/csv/products-valid.csv` on the Imports page as `admin`. The same file twice is ignored (it is deduplicated by hash), so change one price in the file before each run.

Request side (the HTTP thread)

#	Breakpoint	What to look at
1	`src/Catalogue.Web/Endpoints/ImportEndpoints.cs:20` — multipart check	`http.Form.Files[0]` and the `supplierId` form field
2	`src/Catalogue.Application/ImportService.cs:34` — `var hash = ...SHA256...`	The idempotency key; line 37 returns the existing import for a repeated file
3	`ImportService.cs:41` — `storage.SaveAsync(...)`	Where the raw file goes: Blob (Azurite) or `./.storage/files`
4	`ImportService.cs:50` — `queue.PublishAsync(new ImportRequested(...))`	The message: import id, blob path, format, correlation id and trace parent. The HTTP request ends right after this with 202

Worker side (a background thread — the Threads panel shows it)

#	Breakpoint	What to look at
5	`src/Catalogue.Web/LocalImportWorker.cs:26` — `processor.ProcessAsync(message, ...)`	The message arrives through the in-memory queue (`src/Catalogue.Infrastructure/Messaging/ImportQueues.cs:11`); with Service Bus enabled the standalone worker (`src/Catalogue.Workers/ImportWorker.cs:10`) receives it instead
6	`src/Catalogue.Application/ImportProcessor.cs:17` — `CorrelationContext.Begin(...)`	The correlation id from the upload request is restored, so worker logs and audit events link back to it
7	`ImportProcessor.cs:45` — `parsers.FirstOrDefault(p => p.CanHandle(import.Format))`	CSV parser (`src/Catalogue.Infrastructure/Parsing/CsvParser.cs:16`) or EDI parser (`EdiParser.cs:53`)

#	Breakpoint	What to look at
8	`ImportProcessor.cs:67` — `RowValidator.Validate(row)`	**Conditional breakpoint** `parsed.RowNumber == 3` to inspect one row: `row` is the canonical model every format is parsed into
9	`ImportProcessor.cs:74` — `HvacRowValidator.Validate(...)`	Typed spec checks against the category template (`src/Catalogue.Application/HvacImport.cs:43`)
10	`ImportProcessor.cs:80` — `errorBuffer.Add(new ImportItemError(...))`	A rejected row: field, message and raw value, shown later on the import's Errors tab
11	`ImportProcessor.cs:128` — `ApplyBatchAsync`	Every 500 valid rows: products upserted, prices, offers and stock
12	`ImportProcessor.cs:226` — `NeedsPriceApproval(...)`	A price change above the supplier's threshold becomes a change request (line 228) instead of a live price — see tour 6
13	`ImportProcessor.cs:98` — `import.Complete(total, processed, errors)`	Final counts and status: Completed or CompletedWithErrors

Tips: put a **logpoint** at line 65 with `row {total}` to watch progress without stopping. To see a failure path, upload a file for an inactive supplier (exception breakpoint on `InvalidOperationException` stops at `ImportService.cs:19`). Reprocessing (`POST /api/imports/{id}/reprocess`) goes through `src/Catalogue.Application/ImportReprocess.cs` and lands at stop 5 again.

7. Tour 5 — Event Hub streaming of large EDI files

A large EDI file is cut into chunks, the chunks travel as events over hub partitions, and each partition consumer bulk-writes its chunk once. On the Event Hub page as `admin` , generate a small file (1,000 line items) and stream it; small files keep each stop quick.

By default the hub is **in-memory** (`EventHub:Enabled=false`): partitions are channels inside the API process, so every breakpoint below is in one debugger. With the emulator (`docker compose up -d eventhubs` , `EventHub:Enabled=true`) the same code runs against `AzureEventHubProducer` and `EventProcessorClient` .

#	Breakpoint	What to look at
1	`src/Catalogue.Web/Endpoints/EventHubEndpoints.cs:100` — `streams.StartFromLibraryAsync(...)`	The request returns 202 straight after; the work is queued
2	`src/Catalogue.Application/Streaming/EdiStreamService.cs:31` — `StartFromLibraryAsync`	Hash check (an already-streamed file returns `existing: true`), the `ImportFile` with format `EDI-STREAM` , the job put on a channel

#	Breakpoint	What to look at
3	`src/Catalogue.Infrastructure/Streaming/StreamingServices.cs:204` — `Parallel.ForEachAsync(jobs.ReadAllAsync(...))`	The background publisher picks up the job
4	`src/Catalogue.Application/Streaming/EdiStreamPublisher.cs:91` — `producer.SendAsync(pending, ct)`	A batch of chunk events: `pending[0].ChunkIndex`, its line items, the partition key
5	`src/Catalogue.Infrastructure/Streaming/EventHubs.cs:163` (in-memory) or `:90` (Azure) — `SendAsync`	How events are spread over partitions
6	`StreamingServices.cs:47` (in-memory) or `:97` (`ProcessEventAsync`, Azure)	One consumer per partition: open the **Threads** panel to see them run side by side
7	`StreamingServices.cs:157` — `processor.ProcessAsync(chunk, partitionId, ...)`	**Conditional breakpoint** `partitionId == "2"` to follow a single partition
8	`src/Catalogue.Application/Streaming/EdiChunkProcessor.cs:23` — `ProcessAsync`	Parse and validate the chunk's lines, the same validators as tour 4
9	`EdiChunkProcessor.cs:84` — `writer.WriteChunkAsync(...)`	Step in to `src/Catalogue.Infrastructure/Streaming/SqlBulkCatalogueWriter.cs:28`
10	`SqlBulkCatalogueWriter.cs:40` — ledger insert	The chunk ledger: a duplicate (ImportId, ChunkIndex) means "already applied", so a redelivered event changes nothing
11	`SqlBulkCatalogueWriter.cs:94` — `bulk.WriteToServerAsync(...)` and `:135` — `MERGE dbo.Products`	Rows go to a temp table with SqlBulkCopy, then one MERGE per table (products, inventories, offers, specs)
12	`StreamingServices.cs:106` — checkpoint (Azure only)	Checkpoints every 25 events or 5 s, stored in Blob

Prove the idempotency: stream a file, then call `POST /api/eventhub/streams/{id}/restream` with a breakpoint at stop 10: every chunk is applied again only because restream clears the ledger first (`EdiStreamService.cs:59`). Pausing in a consumer while the hub keeps delivering is normal; the other partitions carry on.

8. Tour 6 — Approvals for new products and price changes

A supplier's policy decides whether an imported change goes live or waits as a change request; approving it applies the change. The database already holds about 45,000 pending price changes, so you can start at the Approvals page directly. To create a fresh one, set a supplier's price threshold to 5% on the Suppliers page and import a file where one price rises 10% (tour 4, stop 12).

#	Breakpoint	What to look at
1	`src/Catalogue.Domain/HvacDomain.cs:433` — `NeedsPriceApproval(...)`	During an import: old price, new price and the threshold that decide it
2	`src/Catalogue.Web/Endpoints/ApprovalEndpoints.cs:68` — `approvals.DecideAsync(req.Ids, ...)`	Approve two selected rows on the Approvals page
3	`src/Catalogue.Application/Approvals.cs:21` — `foreach (var chunk in ids.Distinct().Chunk(Batch))`	Work happens in batches of 1,000 to keep each transaction small
4	`Approvals.cs:65` — loop over the requests	One request at a time, oldest first
5	`src/Catalogue.Domain/HvacDomain.cs:423` — `ProductChangeRequest.Decide(...)`	Status, decided-by (from the signed-in user) and note recorded on the request
6	`Approvals.cs:73` — `product.Activate()`	A new product goes live
7	`Approvals.cs:82` — `active?.Close(now)` then line 83 `ProductPrice.For(...)`	A price change: the old price is closed, a new one opened, so price history is kept
8	`ApprovalEndpoints.cs:77` — `DecideMatchingAsync` (button "Approve all")	Returns `remaining`; the SPA calls again until it reaches 0. Put a **logpoint** `decided {decided} remaining {remaining}` on the next line to watch the passes

After a decision the endpoint clears the `products` cache tag, so the next catalogue search (tour 3, stop 3) recomputes counts. The assistant's "approve change" proposals reach stop 2's service through `src/Catalogue.Web/Agent/AgentWeb.cs` (tour 10).

9. Tour 7 — Media: upload, file detection, thumbnails and signed URLs

Files are identified by their bytes, stored privately in Blob storage, and served through short-lived signed links. As `admin`, open a product, choose Edit, and drop a PNG and a PDF onto the media panel.

Upload (`POST /api/products/{id}/assets`)

#	Breakpoint	What to look at
1	`src/Catalogue.Web/Endpoints/MediaEndpoints.cs:63` — `foreach (var file in form.Files.Take(20))`	Each uploaded file in turn; rejected ones are collected, not fatal
2	`src/Catalogue.Application/Media/Media.cs:113` — `MediaSniffer.Detect(bytes)`	Step into line 39: the first bytes decide the type (`89 50 4E 47` = PNG, `%PDF-` = PDF). Rename a `.txt` to `.png` and upload it: rejected here
3	`Media.cs:121` — `var sha = ...SHA256...`	Uploading the same bytes again returns the existing asset (deduplication)
4	`Media.cs:131` — `images.Thumbnail(bytes, ThumbnailEdge)`	Step into `src/Catalogue.Infrastructure/Media/MediaInfrastructure.cs:26`: SkiaSharp renders a 320 px JPEG
5	`Media.cs:138` — `storage.SaveAsync(path, ...)`	Step into `src/Catalogue.Infrastructure/Storage/MediaStorages.cs:23` (Blob) or `:63` (disk): the storage path `products/{id}/{hash}`
6	`Media.cs:143` — `asset.SetPrimary(true)`	The first image becomes the main image

Signed download (`GET /api/media/{id}/t?exp=&sig=`) — reload the catalogue page; every thumbnail is one request.

#	Breakpoint	What to look at
1	`Media.cs:68` — `MediaUrlSigner.Url(...)`	While the product list is built: expiry rounded to 5 minutes, HMAC signature
2	`src/Catalogue.Web/Endpoints/MediaEndpoints.cs:29` — `signer.Verify(...)`	Change one character of `sig` in the browser's address bar: 403 here

#	Breakpoint	What to look at
3	`MediaEndpoints.cs:33` — `If-None-Match` check	Reload: the browser sends the ETag and gets 304 without touching storage

Bulk media pack (`POST /api/media/bulk`) — upload `samples/media/hvac-media-pack-300.zip` on the Media library page. Break at `Media.cs:200` (`MatchAsync(key, ct)`) with a **hit count** of 10, then step into line 223: the file name is matched by GTIN, then MPN, then SKU.

Links from supplier feeds are downloaded in the background by `MediaFetchService` (`MediaInfrastructure.cs:79`). Its SSRF guard at line 114 (`UrlGuard.RejectReasonAsync`, line 48) refuses links that resolve to private or loopback addresses.

10. Tour 8 — Marketplace: compare, basket, order, status

Every seller's offer is priced for the buyer, the basket is priced again on the server, and one purchase order is created per seller. Start **API + SPA**, sign in as `buyer`, open a product and choose the *Compare & buy* tab.

Comparing offers (`GET /api/marketplace/products/{id}/offers`)

#	Breakpoint	What to look at
1	`src/Catalogue.Web/Endpoints/MarketplaceEndpoints.cs:40` — `pricing.CompareAsync(...)`	The buyer's customer account and own supplier id (a company never buys from itself)
2	`src/Catalogue.Application/Marketplace.cs:68` — `var orderQty = RoundUp(...)`	Quantity rounded up to MOQ and pack size: ask for 3 of an item packed in 5
3	`Marketplace.cs:76` — `blocked = "Your own offer"`	Sign in as `seller` (a company that buys and sells): its own offer is excluded here
4	`Marketplace.cs:89` — tier discount	The buyer's price-list discount applied to each seller's price

#	Breakpoint	What to look at
5	`Marketplace.cs:92` — `discounts.TryGetValue(o.SupplierId, ...)`	The seller's own discount for this buyer (HVX gives BriteHeat 6%)
6	`Marketplace.cs:105` — `var best = ...OrderBy(q => q.LineTotal)`	The cheapest offer by line total, not unit price

Basket and order

#	Breakpoint	What to look at
7	Browser: `web/catalogue-ui/src/basket.ts:25` — `add(item)`	Click *Add from HVX*: the basket lives in the browser (localStorage, per user)
8	Browser: `web/catalogue-ui/src/views/BasketView.vue:25` — quote request	The basket page asks the server for prices; it never trusts its own
9	`Marketplace.cs:128` — `SourcingService.QuoteAsync`	Lines merged, the chosen or cheapest offer per line, saving against the content owner
10	`MarketplaceEndpoints.cs:56` — `sourcing.PlaceAsync(...)`	Click *Place orders*
11	`Marketplace.cs:163` — `var quote = await QuoteAsync(...)`	The basket is priced again on the server, so a tampered browser price has no effect
12	`Marketplace.cs:179` — `db.PurchaseOrders.Add(po)`	One purchase order per seller, prices frozen on its lines

Order status — sign in as `hvx` / `hvx12345` (the seller) and accept, then ship, the order on the Orders page.

#	Breakpoint	What to look at
13	`MarketplaceEndpoints.cs:121` — `AccessAsync(o, me, db, ct)`	Step into line 272: Buyer, Seller, Staff or None. None returns 404 so other companies can't tell the order exists
14	`src/Catalogue.Application/Payments.cs:190` — `OrderStatusService.ChangeAsync`	The single place order status changes, used by the Orders page and by approved assistant proposals
15	`src/Catalogue.Domain/MarketplaceDomain.cs:135` — `TransitionTo(next, note)`	The state machine: try *Mark shipped* before *Accept* and watch the `InvalidOperationException` become a 409

Seller portal: as `hvx`, edit an offer's price on the Seller portal page with a breakpoint at `MarketplaceEndpoints.cs:171` (`offer.UpdateBySeller`): availability is derived from stock and lead time. Listing a new product stops at line 201: an existing GTIN makes it another offer on that product instead of a new product.

11. Tour 9 — Payments: simulated checkout, Stripe, webhook and refunds

A payment's amount always comes from the stored order, confirmations are applied idempotently, and a closed order that was paid is refunded through the same gateway. As `buyer`, open a Submitted order on the Orders page and click *Pay* with *Test payment (simulated)*.

#	Breakpoint	What to look at
1	Browser: `web/catalogue-ui/src/views/OrdersView.vue:52` — `pay()`	The chosen provider; Stripe would redirect, the simulator opens the in-app dialog
2	`src/Catalogue.Web/Endpoints/PaymentEndpoints.cs:41` — `payments.StartAsync(...)`	Only the provider comes from the browser
3	`src/Catalogue.Application/Payments.cs:61` — `var reusable = ...`	Click *Pay* twice quickly: the second click gets the same pending payment back

#	Breakpoint	What to look at
4	`Payments.cs:70` — `new Payment(order, provider, ...)`	`payment.Amount` is copied from `order.Subtotal`
5	`Payments.cs:78` — `gateway.StartAsync(new CheckoutRequest(...))`	Simulated: a `sim_cs_…` reference. Stripe: step into `src/Catalogue.Infrastructure/Payments/PaymentGateways.cs:79` to see the form posted to Stripe (amount in pence, Idempotency-Key)
6	`PaymentEndpoints.cs:80` — `"approve" => new GatewayResult(...)`	Click *Approve payment* in the test dialog (`OrdersView.vue:64`)
7	`Payments.cs:97` — `ApplyAsync`	The same method applies simulator results, Stripe status syncs and Stripe webhooks
8	`src/Catalogue.Domain/MarketplaceDomain.cs:235` — `Payment.Succeed`	Returns `false` when already succeeded: a repeated confirmation changes nothing
9	`Payments.cs:112` — `order.MarkPaid(now)`	The order's payment status becomes Paid

Refund: sign in as `hvx` and reject the paid order. Break at `Payments.cs:143` (`SettleClosedOrderAsync`) and `Payments.cs:160` (`RefundAsync`): the successful payment is refunded, and the order is not changed if the provider refuses (that would be a `PaymentException` with status 502).

Stripe (only with test keys set in user-secrets): after paying on Stripe's page the browser returns to `/orders?payment=…`; `OrdersView.vue:81` (`returnedFromCheckout`) asks the server to sync, which reaches `PaymentEndpoints.cs:61` → `Payments.cs:132` (`SyncAsync`) → `PaymentGateways.cs:122` (`FromSession`). With `stripe listen --forward-to localhost:5080/api/payments/webhooks/stripe` running, the webhook arrives at `PaymentEndpoints.cs:123`: step into `StripeWebhook.Verify` (`PaymentGateways.cs:166`) to see the HMAC and the 5-minute replay window.

Concurrency: the database also guarantees one successful payment per order (a filtered unique index); if two attempts ever both succeeded, the second save lands in `Payments.cs:123` (`catch (DbUpdateException)`). The UI can't produce that race, because stop 3 reuses the pending attempt, so step through it in the unit tests instead: debug `PaymentTests` from Test Explorer (`tests/Catalogue.UnitTests/PaymentTests.cs`).

12. Tour 10 — The AI assistant: queue, stream, model, tools, proposals

A question is queued as a background run; the run calls the local model through Agent Framework, executes the tools the model asks for, and streams every step to the browser as Server-Sent Events. Run `foundry server start` first, start **API + SPA**, sign in as `admin` and ask on the Assistant page: *Which orders are waiting for the seller to accept, and did any supplier imports fail?*

Request → queue (HTTP thread)

#	Breakpoint	What to look at
1	`src/Catalogue.Web/Endpoints/AgentEndpoints.cs:108` — `assistant.AskAsync(...)`	The request returns 202 with a `runId` right after
2	`src/Catalogue.Web/Agent/AgentWeb.cs:17` — `AgentPrincipalFactory.CurrentAsync`	Every API policy is evaluated for this user: `policies` is what the agent may do on their behalf
3	`src/Catalogue.Agents/Runs/AgentRuns.cs:111` — `Enqueue`	One run per user, a bounded queue (try asking twice at once: 429)
4	`AgentEndpoints.cs:125` — `await foreach (var e in run.ReadAsync(...))`	The SSE stream: each event written as `id:`, `event:`, `data:`

The run (background thread `AgentRunWorker`)

#	Breakpoint	What to look at
5	`AgentRuns.cs:166` — `await run.Work(run, cts.Token)`	The worker takes one run at a time (one GPU)
6	`src/Catalogue.Agents/CatalogueAssistant.cs:70` — `runtime.EnsureReadyAsync(ct)`	Step into `src/Catalogue.Agents/FoundryLocal/FoundryLocalRuntime.cs:146`: `foundry server status` finds the port, `/v1/models` maps the alias, and line 169 runs `foundry model load` if needed
7	`CatalogueAssistant.cs:76` — `ToolRouter.Select(question, ...)`	Which tools this question gets (`CatalogueAgentTools.cs:405`): an orders question doesn't pay for the import tools

#	Breakpoint	What to look at
8	`CatalogueAssistant.cs:103` — `agent.RunStreamingAsync(...)`	Agent Framework's `ChatClientAgent` starts the model call
9	`src/Catalogue.Agents/FoundryLocal/FoundryLocalChatClients.cs:28` — `Normalize`	Message history before it is sent; tool-call-only turns get text content (Ministral needs it)
10	`FoundryLocalChatClients.cs:138` — `TextToolCallChatClient.GetStreamingResponseAsync`	The raw model text arriving in chunks. With Ministral it begins `list_orders{"status": "Submitted"}`; `LooksLikeBareCall` (`TextToolCallParser.cs:138`) and `TryParseBareCalls` (line 68) turn it into `FunctionCallContent`. With Qwen, `Parse` (line 30) splits `<think>` reasoning from `<tool_call>` XML
11	`src/Catalogue.Agents/Tools/CatalogueAgentTools.cs:21` — `GuardedFunction.InvokeCoreAsync`	Every tool call: arguments, the call budget, errors turned into data, result truncation. Step in once more to reach the tool itself (e.g. `ListOrders`)
12	`CatalogueAssistant.cs:109` — `case TextReasoningContent`	Reasoning and answer text streamed to the browser as events
13	`CatalogueAssistant.cs:155` — `SaveTurnAsync(...)`	Answer, tool trace, token usage and the serialised agent session saved on the conversation

Proposals and approval

Ask *Propose accepting order PO-…* while signed in as `hvx`. Break at `CatalogueAgentTools.cs:233` (`ProposeAction`): nothing is executed, a proposal row is created. Then click *Approve* on the card: `src/Catalogue.Web/Agent/AgentWeb.cs:39` (`DecideAsync`) → line 47 re-checks the **approver's** permissions → line 70 runs the action through the same services as the normal endpoints (tour 8, stop 14).

Tips: set **logpoints** rather than breakpoints at stops 10–12, so the stream keeps flowing and you see the order of events in the Debug Console. A paused run counts against `Agent:RunTimeout` (20 min) and `Agent:FoundryLocal:RequestTimeout` (10 min), and the SPA reconnects to the stream automatically after a long pause.

13. Tour 11 — Operations review workflow and analytics

The review is an Agent Framework workflow of four executors run in-process; analytics is a set of SQL aggregations cached per period. As `admin`, open *Operations review* and click *Run review now*.

Operations review

#	Breakpoint	What to look at
1	`src/Catalogue.Web/Endpoints/AgentEndpoints.cs:184` — `runner.StartAsync("manual", ...)`	A review row is created and a run queued (same queue as tour 10)
2	`src/Catalogue.Agents/Review/OperationsReviewWorkflow.cs:307` — `new WorkflowBuilder(collect)`	The graph: collect → triage → briefing → publish, output taken from publish
3	`OperationsReviewWorkflow.cs:329` — `InProcessExecution.RunStreamingAsync(...)`	The workflow starts; line 334 receives `ExecutorInvokedEvent` / `ExecutorCompletedEvent` for each step and publishes them to the browser
4	`OperationsReviewWorkflow.cs:46` — `CollectSignalsExecutor.HandleAsync`	Deterministic queries over imports, approvals, orders, payments, companies and media: inspect the returned `OperationsSnapshot`
5	`OperationsReviewWorkflow.cs:94` — `TriageExecutor.Triage`	Rules turn the snapshot into findings, with suggested actions (reprocess a failed import, approve a ±2% price change)
6	`OperationsReviewWorkflow.cs:141` — `BriefingExecutor.HandleAsync`	The model writes the briefing by calling `submit_briefing` (line 158); line 168 ends the run as soon as the tool is called. If the model fails or times out, the `catch` returns the rule-based briefing
7	`OperationsReviewWorkflow.cs:245` — `PublishExecutor.HandleAsync`	The review saved and proposals created for suggested actions

The scheduler (`OperationsReviewWorkflow.cs:393` , `TickAsync`) runs every 10 minutes, starts a review when the last one is older than 12 hours and expires old proposals: put a breakpoint there and wait, or restart the API.

Analytics — open the Analytics page and switch between *7 days* and *Year*.

#	Breakpoint	What to look at
1	`AgentEndpoints.cs:219` — `analytics.GetAsync(range, scope, ct)`	Scope: platform staff see everything, a company user only their own orders and payments
2	`src/Catalogue.Application/Analytics.cs:14` — `AnalyticsWindow.For(range, now)`	The period, its buckets (hour, day, week, month) and the previous period used for the change percentages
3	`Analytics.cs:103` — `BuildAsync`	Runs only on a cache miss (2 minutes per range and scope)
4	`Analytics.cs:230` — `Totals_`	One SQL `GROUP BY` per day (or hour) for each series; evaluate `inWindow.ToQueryString()` to see it
5	`Analytics.cs:205` — `ProductTotalsAsync`	Per-day counts for 7.2 million products computed once and cached for 10 minutes, then sliced per period in memory

The first load of a period can take 20–30 seconds; later periods return in under a second, because they reuse the cached product counts.

14. Tour 12 — The Vue frontend

The SPA keeps the access token in memory, adds it to every request, refreshes it on a 401, and guards routes by policy. Launch **Vue: Debug SPA in Edge** (or *API + SPA*); breakpoints set in VS Code on `.vue` and `.ts` files then work like backend ones. Vite serves source maps, so you step through your own TypeScript, not bundled code.

#	Breakpoint	What to look at
1	`web/catalogue-ui/src/main.ts` — the `await restoreSession()` line	On a page reload the session is rebuilt from the refresh cookie before the app mounts
2	`web/catalogue-ui/src/api.ts:24` — `restoreSession` and `:15` — `refreshSession`	`refreshSession` is single-flight: ten parallel 401s cause one refresh
3	`web/catalogue-ui/src/router.ts:41` — `router.beforeEach`	The policy check per route: sign in as `buyer` and type `/admin/organizations` in the address bar
4	`web/catalogue-ui/src/session.ts:47` — `can(policy)`	The policy table mirrored from the API; it only hides UI, the API still enforces
5	`web/catalogue-ui/src/api.ts:34` — request interceptor	The `Authorization: Bearer …` header added to each call
6	`web/catalogue-ui/src/api.ts:45` and `:50` — response interceptor	A 401: refresh once and replay; 408/429/5xx on GET: retry with backoff, honouring `Retry-After`
7	`web/catalogue-ui/src/views/CatalogueView.vue:102` — `watch([...], reload)` and `:66` — `load()`	Each filter change triggers a reload; inspect the params sent to `/api/products`
8	`CatalogueView.vue:185` — `addToBasket(o)`	Tour 8's basket entry point
9	`web/catalogue-ui/src/views/AssistantView.vue:51` — `send()` and `:78` — `onEvent(e)`	Tour 10 from the browser side: each SSE event updating the live bubble
10	`web/catalogue-ui/src/api.ts:320` — `followRun`	The stream reader: replays from the last event id after a dropped connection, refreshes the token on 401 (line 326)

DevTools alongside the debugger: the *Network* tab shows each request's `X-Correlation-Id` response header, the same id as in the API logs and audit events (tour 1). Install the Vue DevTools extension to inspect component state, such as `report` on the Analytics page or `live` on the Assistant page.

15. Troubleshooting and power tips

Problem	What to do
Breakpoint shows as hollow ("not bound")	The build is stale or the file is not part of the running process: stop, rebuild (the launch configs run `build` first), and check you attached to `Catalogue.Web`, not the gateway
A handler breakpoint never hits	It is probably on the `MapGet(...)` line rather than inside the lambda (section 2), or the request was rejected earlier: 401/403 by authorization, 429 by a rate limit (login: 10 per minute)
You hit the same breakpoint twice	The SPA retried a GET after its 60 s timeout; drive the request from Swagger or `Catalogue.Web.http`
`OperationCanceledException` after a long pause	A timeout outside the API fired: the gateway (60 s), the Vite proxy or the browser. Go to the API directly on :5080
401 after a long session	The 30-minute access token expired; the SPA refreshes it, Swagger needs a new login
Want to see the SQL	At a breakpoint, evaluate `query.ToQueryString()` on any `IQueryable`. For all SQL, temporarily set the EF Core override in `src/Catalogue.ServiceDefaults/ServiceDefaults.cs:72` to `LogEventLevel.Information` (code overrides win over appsettings here)
Want to see what EF will save	Evaluate `((DbContext)db).ChangeTracker.DebugView.LongView` before `SaveChangesAsync`

Problem	What to do
Background work you can't find	Open the **Threads** panel: `LocalImportWorker`, `EdiStreamPublisherService`, the partition consumers, `AgentRunWorker`, `MediaFetchService` and `OperationsReviewScheduler` each run on their own
Following one request across logs, audit and traces	Copy `X-Correlation-Id` from the response: search for it in `src/Catalogue.Web/logs`, in the Audit log page and in the Aspire dashboard (`http://localhost:18888`, needs `OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317`)
Exceptions vanish into a problem response	Turn on *All Exceptions* in the Breakpoints panel; the debugger stops where it is thrown, before `GlobalExceptionHandler`
Need to stop on one record only	Conditional breakpoint on its id (`product.Id == 1234`, `message.ImportId == 7`, `partitionId == "2"`)
The assistant says the model is unavailable	`foundry server status`; if it is stuck, `foundry server restart`, then `foundry model load ministral-3-3b-instruct-2512`
Debugging a unit test	Test Explorer → right-click the test → *Debug Test*. Tests run against EF Core in-memory and fakes, so they are the fastest way to step through pricing, payments, parsing and the agent's tool-call parser

Suggested order for learning the codebase: tours 1, 2 and 3 first (every request depends on them), then 8 and 9 for the business flow, 4 and 5 for data ingestion, and 10 and 11 for the AI parts. The endpoint-by-endpoint reference is in `docs/api/` in the repository.